

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Web-System Architecture of Your Own Project

Web-System Architectures and End-to-End Data Flow

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 2
Output	System-specific architecture diagram, request-to-response trace, and group presentation
Maximum Score	80 points

Student / Project Information

Block: <u>3A</u>	Date: <u>September 9, 2026</u>
Group No.: <u>DaisyMeRollin</u>	Instructor: <u>Jean Y. Villasin, MIT</u>
System Title: SariwaBa?: Fish Freshness Classification System using Machine Learning and Mobile Computer Vision	
Members: Marc Elson M. Rebato, Benedict A. Mendoza	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 2 **1. Activity Alignment and**

Purpose

Topic Learning Outcome: Explain common web-system architectures and trace the end-to-end flow of data through major system components.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Web-system architecture diagram and request-to-response tracing exercise.

Purpose of the Activity

Before writing major code, your team must create an architecture blueprint that explains how the users, Flutter interface, application or API layer, database, external services, security controls, and deployment environment work together. The diagram will serve as a technical guide throughout the semester project.

Important distinction: Flutter is the client-side application framework. It is one part of the complete web system; your architecture must also show how data is processed, stored, secured, and returned to the user.

2. Intended Learning Objectives

- Identify the users, components, layers, services, and deployment resources required by the team's proposed system.
- Select and justify an appropriate architecture such as two-tier, three-tier, monolithic, service-oriented, or a defensible hybrid architecture.
- Create a clear system-specific architecture diagram using accurate component names, directional arrows, protocols, and data labels.
- Trace one important workflow from the user action through the Flutter client, backend/API, business logic, data store, and response shown to the user.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and be able to explain the diagram.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One system architecture diagram based on the team's actual approved project.
Supporting Output	One request-to-response trace, a 100–150 word architecture justification, and a group presentation.
Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, or an equivalent diagramming tool.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned architecture component or data-flow step.

Requirements

- Approved or proposed system title and a brief statement of the community or organizational problem being addressed.
- List of primary users or roles and at least three major features of the system.
- Tentative technology stack, including Flutter, backend/API or backend-as-a-service, database, external services, and deployment platform.
- A copy of the lecture notes on web-system layers, two-tier and three-tier architectures, monolithic and service-oriented architectures, and request-response flow.

5. Part A – System Component Inventory

Complete the worksheet before drawing. Use the exact names planned for your own project. Do not write only generic labels such as “front end” or “database” when a more specific component name is already known.

Architecture Item	Your System	Notes / Examples of Evidence
System title and purpose	Fish Freshness Classification System using Machine Learning and Computer Vision	Solves the difficulty of assessing fish freshness subjectively in wet markets and household settings by providing an automated image classification web and mobile tool.
Actors / user roles	Consumers, Fish Vendors	Primary users who upload or capture fish photos, execute freshness classifications, and view their past scan logs.
Flutter client	Flutter Web Application, Flutter Mobile Application (Android/iOS)	Live Camera View, Image File Picker, Classification Display, and History Records Screen
State and client logic	Riverpod State Management, Client-side Image Validator	Manages scan session state, handles file size/format checking (.jpg, .png), and stores user auth tokens in memory.
Backend / API / BaaS	Django REST Framework API	Central REST API providing routing endpoints: /api/v1/classify/, /api/v1/auth/google/, and /api/v1/history/
Business logic	Image Preprocessing Pipeline, Classification Service, History Service	Validates image bytes, converts images into normalized RGB tensors, invokes the model, formats confidence scores, and triggers persistence.
Database and storage	PostgreSQL Database	Stores application schemas: users, classification_history, and fish_images
External services	Google Authentication Services, Machine Learning Classification Services	Google handles passwordless identity verification; the decoupled ML model processes visual tensors to classify freshness.
Security controls	Google Token Verification Middleware, Backend Validation, Input Validation	Authenticates Google ID token signatures, enforces object-level authorization on scan records, and sanitizes file uploads.

6. Part B – Select the Architecture Style

Select the architecture that best fits your system. More than one may apply, but the final choice must be justified based on requirements—not merely because it is familiar.

Architecture	Core Structure	When It May Fit	Select
Two-tier	Client communicates directly with a data service or tightly coupled server.	Small systems with controlled scope; security and maintainability must still be addressed.	☐
Three-tier	Flutter presentation → application/API → database.	Recommended for systems that need clear separation of responsibilities and secure data access.	☐

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 3 of 9

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 2

Architecture	Core Structure	When It May Fit	Select
Monolithic	Application modules are packaged and deployed as one backend application.	Small to medium systems requiring simpler deployment and coordinated modules.	☐
Service-oriented / Hybrid	Multiple services, BaaS modules, or external APIs cooperate.	Systems that genuinely need independent services such as GIS, ML, notifications, or payments.	☐

Selected architecture:

Three-tier Architecture

Initial justification (3–5 sentences):

SariwaBa adopts a three-tier architecture to decouple the client presentation, application processing, and persistent database into isolated, maintainable layers. A single shared Django REST Framework backend serves both the Flutter Web and Flutter Mobile clients, preventing duplicate backend development while enforcing centralized authorization and input validation. The PostgreSQL database remains strictly protected behind the application tier and is never accessed directly by client applications. Additionally, externalizing user authentication to Google OAuth and decoupling the ML inference service preserves clear modular boundaries and simplifies future system updates.

7. Part C – Construct the Web-System Architecture Diagram

Create one clean, readable diagram that shows the complete architecture of your own system. The diagram must be understandable even to a reviewer who has not seen your code.

No.	Required Element	What Must Be Shown
1	System identity	Place the official system title and selected architecture style at the top.
2	Actors	Show every major user role or external actor that sends requests to the system.
3	Client / presentation layer	Show Flutter Web or Flutter mobile, key screens or UI modules, routing, state management, and form validation as appropriate.
4	Communication layer	Show how components communicate: HTTPS, REST, WebSocket, SDK, JSON, or another relevant mechanism.
5	Application / API layer	Show request handlers, authentication, authorization, validation, service/business logic, and major server modules.
6	Data layer	Show the selected database, important tables/entities or data domains, file storage, relationships, and constraints at an appropriate level.
7	External services	Show only services actually used by the project, such as GIS/maps, ML model or API, email/SMS, payment, weather, or cloud storage.
8	Deployment	Show where the Flutter client, backend/API, database, and storage will run or be hosted.
9	Data flow	Use directional arrows and label important requests, responses, protocols, data formats, and returned results.
10	Security	Show where login, role checks, tokens, input validation, access control, and protected resources are handled.
11	Legend	Use consistent symbols, colors, line styles, and a brief legend.

Minimum Diagram Rule

Your architecture must show at least one complete path: Actor → Flutter interface → API/application component → database or external service → response returned to the Flutter interface. Every arrow on this path must have a meaningful label.

8. Reference Example – Do Not Copy as Your Final Output

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 2

The example below illustrates the level of clarity expected in a Flutter-based architecture. It is not a template to copy. Replace it with the actual users, features, services, data, security controls, and deployment choices of your own system.

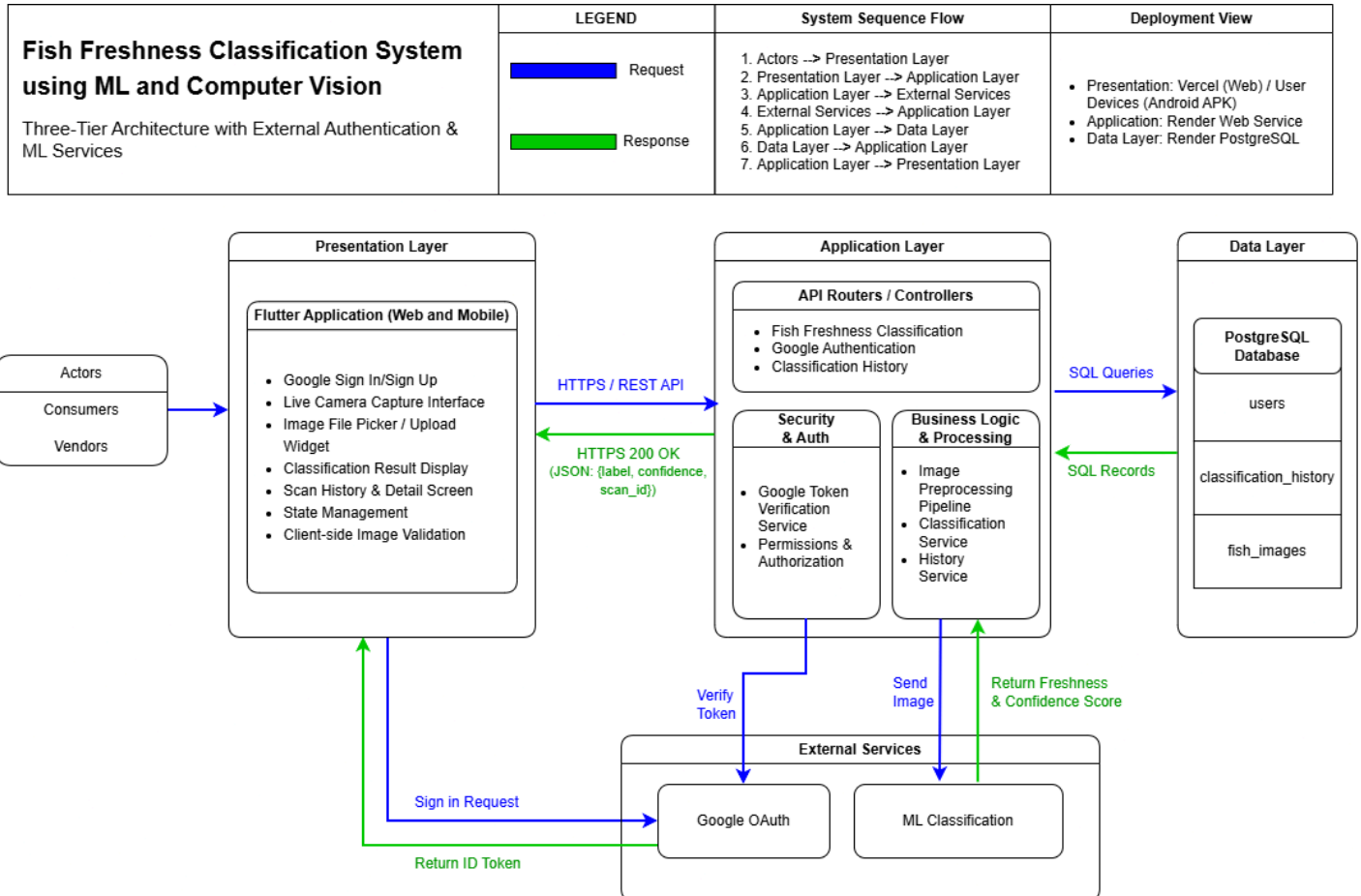


Figure 1. Example of a complete request-and-response path in a Flutter-based three-tier architecture.

1. User action	2. HTTPS / JSON request	3. Query or transaction
4. Records or result	5. HTTP status + JSON	6. Updated Flutter interface

9. Recommended Architecture Labels for Flutter Projects

Layer / Area	Possible Label	What It Represents
Presentation	Flutter Web / Flutter App	Screens, widgets, forms, navigation, and user feedback
Client state	Provider / Riverpod / BLoC / controller	Coordinates UI state and user actions
Communication	HTTP client, SDK, or repository/service class	Sends requests and receives responses
Application	REST API, BaaS, serverless function, or custom backend	Routes requests, validates data, and applies business rules
Security	Auth service, token verification, role checks	Protects accounts, data, and restricted operations
Data	PostgreSQL, storage, and important entities	Persists system records and files

10. Laboratory Procedure

Phase 1 – Analyze the system (20 minutes)

Review the approved title, problem, users, major features, and available technology choices. Complete the component inventory.

Phase 2 – Choose the architecture (20 minutes)

Select the architecture style and write an initial justification based on security, maintainability, scalability, integration, and deployment.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 2 Phase 3 – Draft the diagram (45 minutes)

Arrange the actors and layers from left to right or top to bottom. Add all major system components before drawing arrows.

Phase 4 – Label data flow (35 minutes)

Label requests and responses using actual workflows, protocols, endpoints or service names, data formats, status/result, and security checks.

Phase 5 – Trace one key transaction (30 minutes)

Use the table in Part D to trace one important user action through the complete architecture.

Phase 6 – Review and improve (15 minutes)

Use the quality checklist. Ask another group or the instructor to identify missing components, unclear arrows, or incorrect layer responsibilities.

Phase 7 – Export and submit (15 minutes)

Export the final diagram, complete the justification, commit the files to GitHub, and submit the required files through MasaoLMS.

11. Part D – Request-to-Response Tracing Exercise

Select one critical workflow from your own system. Examples include login, submitting a report, registering a boarding house, approving a request, uploading an image, requesting a prediction, or retrieving a map-based recommendation.

Selected workflow:

Fish Freshness Image Capture, Classification, and History Persistence

Expected final result shown to the user:

Updated Flutter UI displaying the freshness verdict badge ("Fresh" / "Not Fresh"), confidence score percentage, and an entry saved to scan history

Step	Actor / Component	Action or Data	Protocol / Mechanism	Expected Result
1	Actor (Consumer/Vendor)	Captures a fish photo or uploads an image file	UI Touch / Camera Device Event	Raw image file loaded into Flutter application memory
2	Flutter Client	Checks file format (.jpg/.png) and file size bounds	Client-side validation logic	Image verified; UI displays loading spinner via Riverpod state
3	Flutter Client	Packages image with cached Google ID token	HTTPS POST /api/v1/classify/ (multipart/form-data)	HTTP classification request dispatched across the network
4	Django Security & Auth	Verifies token signature, issuer, and expiry against Google	Google Token Verification Middleware	Request authenticated; rejects with 401 Unauthorized if invalid
5	Django Application Tier	Verifies endpoint access permissions	Django Backend Auth Validation	Identity authorized to access classification resource
6	Django Business Logic	Decodes image bytes, resizes, and normalizes pixels into a tensor	Preprocessing Pipeline	Normalized RGB tensor array prepared for the model
7	ML Classification Service	Executes computer vision inference on the input tensor	Internal IPC / Python Service Call	Returns freshness label (Fresh/Not Fresh) and confidence score
8	Django Business Logic	Formulates SQL insert query with user ID, label, and score	Django ORM / PostgreSQL Driver	INSERT INTO classification_history dispatched
9	PostgreSQL Database	Executes write transaction and indexes records	SQL Transaction Engine	New scan record ID returned to Django ORM
10	Flutter Client	Django responds with HTTPS 200 OK; Riverpod resolves state	JSON Response ({label, confidence, scan_id})	Classification Result Display renders freshness badge to the actor

Tracing Guidance

A strong trace normally includes user input, Flutter-side validation, construction of the request, authentication or token checking, server-side validation, business-rule execution, database or external-service access, response generation, Flutter state update, and user feedback.

12. Part E – Architecture Justification

Write 100–150 words explaining why the selected architecture is appropriate for your system. Address all of the following:

- How the architecture supports the system's main users and features.
- Why the responsibilities are divided among the Flutter client, application/API layer, and data layer.
- How the design supports security, maintainability, scalability, integration, and deployment.
- Any limitation or risk that the team must manage during implementation.

SariwaBa utilizes a three-tier architecture with supporting external components to provide a unified backend for both Flutter Web and Mobile clients. The presentation tier manages UI interactions and client-side validation using Riverpod, while the application tier centralizes request validation, authorization, and computer vision coordination through Django REST Framework. The data tier securely isolates persistent user profiles and scan records within PostgreSQL. Externalizing identity verification to Google OAuth eliminates the risk of storing passwords locally, allowing Django to focus strictly on route permissions and data authorization. Decoupling the machine learning classification model from the persistence layer ensures the model can be retrained or updated without breaking database schemas or business rules. The primary implementation risk is inference latency under constrained network bandwidth; the team mitigates this by enforcing strict client-side image compression prior to API transmission.

13. Final Quality Checklist

☐ The diagram uses the team's official system title and actual project features.	☐ Deployment locations or platforms are identified.
☐ The selected architecture style is clearly named and justified.	☐ Request and response arrows have clear directions and meaningful labels.
☐ All major actors or user roles are present.	☐ At least one complete request-to-response path is visible.
☐ The Flutter client and important UI/client responsibilities are shown.	☐ Authentication, authorization, validation, and protected resources are represented.
☐ The application/API or backend-as-a-service components are shown.	☐ The diagram has a legend, readable text, consistent symbols, and no crossing arrows that cause confusion.
☐ The database, file storage, and important data domains are shown.	☐ The request-to-response table agrees with the diagram.
☐ External services are included only when the project actually uses them.	☐ Every group member can explain an assigned component, trace the complete workflow, and answer questions without reading a script.

14. Submission Requirements

- PDF or PNG of the final architecture diagram in readable landscape orientation.
- Editable diagram source file, such as .drawio, .pptx, .fig, or the native file of the selected tool.
- Completed request-to-response tracing table and 100–150 word architecture justification.
- Files committed to the project GitHub repository under a folder such as docs/architecture/.
- Repository link and required files submitted through MasaoLMS using the naming convention: IT110_LA2_Group##_SystemName.

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 7 of 9

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 2

- Deliver a 5–7 minute group presentation using the final architecture diagram as the main visual, followed by a 2–3 minute question-and-answer period. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and design decisions. AI tools may be used to clarify concepts, but the final architecture, labels, request flow, and justification must be reviewed, understood, and defended by the team. The instructor may ask any member to explain a component or trace a request without assistance.

15. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate.

For the presentation criterion, points may be reduced for unclear explanation, unequal participation, excessive reading, incorrect tracing, poor time management, or inability to answer questions about the team's own design.

Criterion	Evidence for Full Credit	Points
Architecture correctness and selection	The selected architecture is appropriate for the system requirements, accurately represented, and justified using clear architectural reasoning.	15
System-specific components and layers	Actors, Flutter client, application/API, business logic, data layer, external services, security controls, and deployment are complete and use the team's actual project labels.	12
Request-to-response and data flow accuracy	Arrows, protocols, request/response direction, validation, processing, data access, and returned results form a complete, consistent, and technically coherent workflow.	15
Security and data protection	Authentication, authorization, role checks, input validation, protected resources, data access controls, and appropriate trust boundaries are clearly represented.	8
Technology-stack integration and deployment	The diagram clearly shows how Flutter, the selected backend/API or BaaS, database, storage, external services, and deployment platforms work together.	8
Visual clarity and technical notation	The diagram uses readable text, consistent symbols, meaningful labels, a useful legend, balanced spacing, and directional arrows with minimal visual confusion.	7
Written architecture justification	The 100–150 word explanation clearly connects the selected architecture to the system's users, features, security, maintainability, scalability, integration, deployment, limitations, and risks.	5

Group presentation and oral defense	The presentation is clear, organized, and completed within the assigned time; the final diagram is used effectively; every member participates; the group accurately explains the architecture and request-to-response trace; and questions are answered correctly and confidently.	10
--	---	-----------

Raw Score	/ 80	Equivalent	_____
Checked by	_____	Date	_____
Remarks			

16. Post-Laboratory Reflection

Which component should contain most of the business rules in your system, and why?

The Application Tier (Django backend) should contain most of the business rules. Because we are using Flutter for both web and mobile, keeping core rules—such as image format checks, preparing images for the ML model, and determining what scan data to save—in Django ensures consistent logic for both platforms so we do not have to write or maintain duplicate rules in Flutter.

What should the system do when the API, database, or internet connection becomes unavailable?

- API Unavailable: Flutter should catch HTTP error network timeouts and present a user-friendly error card prompting them to retry rather than crashing.
- Database Unavailable: Django must catch database connection errors within a try-except block, log the failure internally, and return an error with a message.
- Lost Internet: Flutter should detect the network drop and show a simple message or alert informing the user that an active connection is needed to classify an image.

Which part of the architecture is most security-sensitive?

The Security & Auth module in the Application Layer is the most sensitive. It serves as the gateway to the rest of the system. If token verification is bypassed or configured incorrectly, anyone could access private user scan records or trigger heavy model classifications without authorization.

How would the architecture change if the number of users or stored records increased significantly?

- Application Layer: We would upgrade the cloud server specs and separate the ML model onto its own dedicated server so heavy image processing does not slow down standard API requests.
- Data Layers: We would add database indexes on fields like user ID and date to keep history queries fast, and store uploaded fish images in an external cloud file storage (like Cloudinary or AWS S3) rather than storing large image files directly in the database.

Which component or feature should your team implement first after this activity?

Our team should first implement the basic Flutter-to-Django image upload endpoint. Testing an image upload from Flutter and returning a simple mock classification response from Django confirms that the client and backend communicate properly over HTTPS before we integrate the ML model and database storage.